



Hi-PART: Going Beyond Graph Pooling with Hierarchical Partition Tree for Graph-Level Representation Learning

YUYANG REN, HAONAN ZHANG, LUOYI FU, SHIYU LIANG, LEI ZHOU,

XINBING WANG, and XINDE CAO, Shanghai Jiao Tong University, China

FEI LONG, State Key Laboratory of Media Convergence Production Technology and Systems, Xinhua News Agency, China

CHENGHU ZHOU, Institute of Geographical Sciences and Natural Resources Research, Chinese Academy of Sciences, China

Graph pooling refers to the operation that maps a set of node representations into a compact form for graph-level representation learning. However, existing graph pooling methods are limited by the power of the Weisfeiler–Lehman (WL) test in the performance of graph discrimination. In addition, these methods often suffer from hard adaptability to hyper-parameters and training instability. To address these issues, we propose Hi-PART, a simple yet effective graph neural network (GNN) framework with Hierarchical Partition Tree (HPT). In HPT, each layer is a partition of the graph with different levels of granularities that are going toward a finer grain from top to bottom. Such an exquisite structure allows us to quantify the graph structure information contained in HPT with the aid of structural information theory. Algorithmically, by employing GNNs to summarize node features into the graph feature based on HPT's hierarchical structure, Hi-PART is able to adequately leverage the graph structure information and provably goes beyond the power of the WL test. Due to the separation of HPT optimization from graph representation learning, Hi-PART involves the height of HPT as the only extra hyper-parameter and enjoys higher training stability. Empirical results on graph classification benchmarks validate the superior expressive power and generalization ability of Hi-PART compared with state-of-the-art graph pooling approaches.

CCS Concepts: • **Computing methodologies** → *Artificial intelligence*; • **Networks** → *Network algorithms*;

Additional Key Words and Phrases: Graph representation learning, structural entropy

This work was supported by NSF China (No. 62020106005, 61960206002, 42050105, 62061146002), Shanghai Pilot Program for Basic Research - Shanghai Jiao Tong University. Moreover, this work is also a contribution to the Deep-time Digital Earth (DDE) Big Science Program.

Authors' addresses: Y. Ren, H. Zhang, L. Fu, S. Liang, L. Zhou, X. Wang, and X. Cao, Shanghai Jiao Tong University, 800 Dong chuan Rd, Minhang Qu, Shanghai Shi, China, 200240; e-mails: {renyuyang, zhanghaonan, yiluofu}@sjtu.edu.cn, liangshiyu513@gmail.com, {zhoulei1588, xwang8, xdcao}@sjtu.edu.cn; F. Long, State Key Laboratory of Media Convergence Production Technology and Systems, Xinhua News Agency, 9 Beizhenghongqi West St, Haidian Qu, Beijing Shi, China, 100803; e-mail: longf05@tsinghua.org.cn; C. Zhou, Institute of Geographical Sciences and Natural Resources Research, Chinese Academy of Sciences, 11 Datun Rd, Chaoyang Qu, Beijing Shi, China, 100101; e-mail: zhouch@lreis.ac.cn. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1556-4681/2024/02-ART94 \$15.00

<https://doi.org/10.1145/3636429>

ACM Reference format:

Yuyang Ren, Haonan Zhang, Luoyi Fu, Shiyu Liang, Lei Zhou, Xinbing Wang, Xinde Cao, Fei Long, and Chenghu Zhou. 2024. Hi-PART: Going Beyond Graph Pooling with Hierarchical Partition Tree for Graph-Level Representation Learning. *ACM Trans. Knowl. Discov. Data.* 18, 4, Article 94 (February 2024), 20 pages. <https://doi.org/10.1145/3636429>

1 INTRODUCTION

In recent years, **Graph Neural Networks (GNNs)** [39] have been extensively used for graph-structured data. Several variants of GNNs have been proposed to achieve exceptional performance for different graph-related tasks such as node classification [13] and link prediction [37]. Although GNN has demonstrated its strength in node representation learning, there is a huge gap between node and graph representation learning. While the majority of existing GNNs focus on encoding the nodes of a graph in an accurate and expressive manner, graph pooling methods are proposed to encode the set of node representations to the graph-level representation. It is shown that graph pooling methods achieve state-of-the-art performance in graph-level tasks such as graph classification [29, 36].

Existing graph pooling methods mainly fall into two classes: node clustering [34, 36] and node sampling [11, 38]. Node clustering methods create compressed graphs with super-nodes and learn the cluster assignment matrix to softly map nodes to clusters. Node sampling methods, such as TopKPool [11], rank the nodes in a graph based on the learnable scores and sample top-k nodes to form the sampled graph. Both these methods add a graph pooling layer after each GNN layer to iteratively lower the size of graphs. Finally, a readout function, e.g., summation or maximization, is used to obtain the embedding of the entire graph.

However, existing graph pooling methods do not fully mine the hidden structural information, leading to the following issues:

- **Expressivity.** They may generate the same representation for two different graphs that can be discriminated by the **Weisfeiler–Lehman (WL)** graph isomorphism test [30] as they sub-optimally preserve information from the node features. Recently, Baek et al. [3] proposed the Graph Multiset Transformer, which is shown to be as powerful as the WL test. Nevertheless, all of them operate on the set of node representations learned by GNNs, while the WL test is the upper limit of GNN’s expressive power [31]. Therefore, none of these methods can go beyond the power of the WL test.
- **Sensitivity.** They are sensitive to hyper-parameters involved in graph pooling layers. For different graphs in a dataset, the ratio and number of times of reduction in graph size are not naturally determined by the individual graph structure but artificially set as fixed values. Both have a non-negligible impact on the performance [14].
- **Stability.** The training process might become unstable when supervised by a single classification loss because of the concurrent optimization of the hierarchical structure and graph representation. In node clustering methods such as DiffPool [34], it is difficult to push the pooling GNN away from spurious local minima early in training where additional objectives such as link prediction and entropy regularization are required. In node sampling methods, the sampled nodes might be isolated without any connectivity [11], which will inevitably hurt the performance.

To overcome the above limitations, we propose a simple yet effective GNN framework leveraging graph’s **Hierarchical Partition Tree (HPT)**, termed Hi-PART. The core of Hi-PART is the introduction of HPT, which can quantify the hidden structural information of graph in structural information theory [15]. In HPT, each layer represents a different granularity partition of

the graph, and the most tightly related nodes (sets) are grouped into a higher granularity module, i.e., their parent node. Specifically, we first design a greedy algorithm to optimize the HPT that preserves the maximum possible graph information. Then based on the hierarchical structure of HPT, GNNs are used to summarize the node features to the root node feature (or the graph feature) in a bottom-up way. Through the introduction of HPT, the three issues noted earlier can be resolved in the following manner: (i) with the hierarchical structural information provided by HPT, Hi-PART is theoretically demonstrated to go beyond the power of the WL test in distinguishing non-isomorphic graphs. (ii) Given different graphs, the ratio and times of graph reduction are naturally determined by their HPT structures. (iii) By separating the HPT optimization from graph representation learning, Hi-PART relies on a single classification loss to ensure training stability.

Our main contributions are summarized as follows:

- We propose a simple yet effective framework, Hi-PART, to substitute the graph pooling layers for graph representation learning. Hi-PART only relies on only one additional hyperparameter and has high training stability.
- To the best of our knowledge, Hi-PART is the first method that explicitly applies the structure entropy to graph representation learning. By optimizing structure entropy of the graph and obtaining its HPT, Hi-PART enables a new direction of leveraging tree structures to guide GNN learning graph representation.
- We theoretically demonstrate that Hi-PART’s power in discriminating different graphs goes beyond the WL test, which is the bottleneck of the existing graph pooling methods.
- We conduct experiments on graph classification benchmarks to validate our theoretical analysis and the superior performance of Hi-PART over the SOTA graph pooling methods. The source code is available at: <https://github.com/ryy980622/Hi-PART>.

Organization. The rest of this article is organized as the following. Sections 2 and 3 introduce the related research works and background knowledge. We then elaborate on our proposed Hi-PART in Section 4. Theoretical analysis of Hi-PART’s expressive power is summarized in Section 5. The experimental studies are presented in Section 6, followed by concluding remarks in Section 7.

2 RELATED WORK

2.1 Graph Pooling

Graph pooling methods can be mainly divided into two categories: node clustering and node sampling. DiffPool [34] was the first node clustering method, which used graph pooling layers to cluster nodes into super-nodes by learning a differentiable soft cluster assignment for nodes at each layer of a deep GNN. EigenPool [17] was a pooling operator based on local graph Fourier transform, which can extract subgraph information utilizing both node features and structure of the subgraph. StructPool [36] proposed to formulate the node cluster problem as a structured prediction problem and employs conditional random fields to capture the relationships among the assignments of different nodes. ASAP [22] was a sparse and differentiable pooling method based on a self-attention network along with a modified GNN formulation. MinCutPool [4] employed GNN to compute cluster assignments by optimizing a continuous relaxation of the normalized mincut problem. CGIPool [21] maximized the mutual information between the input and the coarsened graph of each pooling layer to preserve graph dependencies.

The node sampling methods mainly select the most important nodes to form a new graph. DGCNN [38] was a localized graph convolution model connected with a SortPooling layer which sorted graph vertices consistently. G-U-Net [11] used graph pooling and unpooling operations as the encoder and decoder of graphs. SAGPool [14] was a graph pooling method based on self-attention where the self-attention mechanism was exploited to distinguish between the nodes

that should be dropped or retained. LCPN [25] used a local capsule pooling to alleviate the issue of insufficient clustering and a task-aware readout mechanism to obtain a more expressive graph representation.

Apart from these two kinds of methods, EdgePool [8] iteratively chose edges based on the learned scores and pooled the connected nodes. SOPool [29] collected second-order statistics that encode feature correlations and topology information to leverage all node information. Graph Multiset Transformer [3] uniquely embedded two different graphs into two distinct embeddings based on multi-head attention, which was proved to be as powerful as the WL test.

2.2 Quantifying Structural Information

Quantifying structural information is a critical but difficult challenge in information theory. Raszewsky [23] first proposed the entropy measure for graph data. Following this work, several other notions emerged such as Gibbs entropy [1], Shannon entropy [24], and Von Neumann entropy [6]. Intrinsically, all of these measures are the Shannon entropy applied to different types of distribution. In recent years, Li and Pan [15] defined the structure entropy of a graph as the minimum number of bits to encode the vertex that is accessible from a step of random walk [10]. By encoding graphs into HPTs, structure entropy transforms the complex graph system into a simplified form while maintaining crucial features.

In this article, we apply structure entropy as a criterion to build informative HPTs and combine them with GNNs to substitute the graph pooling layers for graph representation learning.

3 PRELIMINARIES

3.1 Graph Neural Networks

We begin by outlining the general framework of GNN. Let $G = (V, E)$ represent a graph with node features X_v for $v \in V$. GNNs use the graph structure and raw node features to learn a deep representation vector h_v for each node $v \in V$ or h_G for the entire graph G if a readout layer is included. Current GNNs follow a two-step strategy containing neighborhood aggregation and combination to iteratively update node features. Formally, the feature updating process in the l th layer of a GNN is as follows:

$$h_v^{(l)} = \phi^{(l)} \left(h_v^{(l-1)}, f^{(l)} \left(\{h_u^{(l-1)} : u \in \mathcal{N}(v)\} \right) \right) \quad (1)$$

Here, $h_v^{(l)}$ is the feature of node v in the l th layer and $\{h_u^{(l-1)} : u \in \mathcal{N}(v)\}$ is the multiset (a set allows possibly repeating elements) of the neighboring node features. f and ϕ are the aggregation and combination functions, respectively, which differ amongst GNNs. For example, in **Graph Convolutional Network (GCN)** [13], the aggregation and combination functions are integrated as follows:

$$h_v^{(l)} = \psi \left(W \cdot \text{MEAN} \{h_u^{(l-1)}, \forall u \in \mathcal{N}(v) \cup \{v\}\} \right) \quad (2)$$

where ψ is the activation function. In **Graph Isomorphism Network (GIN)** [31], the **multi-layer perceptron (MLP)** is used to fit the composition of aggregation and combination functions:

$$h_v^{(l)} = \text{MLP}^{(l)} \left(\left(1 + \epsilon^{(l)} \right) \cdot h_v^{(l-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(l-1)} \right) \quad (3)$$

After L layers of iteration, the graph feature h_G is calculated as follows:

$$h_G = \text{READOUT} \left(\{h_v^{(L)} : v \in G\} \right) \quad (4)$$

The READOUT function can be any permutation-invariant function such as summation, maximization, or average.

3.2 Weisfeiler–Lehman Test

We briefly introduce the WL graph isomorphism test [30] to establish the groundwork for later theoretical research. Graph isomorphism problem aims to judge if two graphs have the same topology. However, no existing polynomial-time algorithm can solve this problem [2]. The WL test is an effective algorithm to discern non-isomorphic graphs except for some corner examples [7]. Concretely, it comprises a two-step iterative process: for each node, (1) aggregate the labels from its neighborhood, and (2) use a hash function to map its label and the aggregated labels to a unique new label. The WL test decides two graphs as non-isomorphic once they have different label distributions in any iteration. It is noteworthy that this two-step process is strikingly similar to the aggregation and combination process in GNNs. Thus Xu et al. [31] have investigated the link between GNNs and the WL test upon which we build our theoretical analysis later.

4 PROPOSED FRAMEWORK

In this section, we provide a detailed introduction to our proposed framework, Hi-PART. The key novelty of Hi-PART is the use of HPT. Here, we first discuss our motivation for using HPT in graph-level representation learning and provide its formal definition. Given a complex graph system consisting of many objects and relationships, it is difficult to derive its high-level representation directly from node representations. Nevertheless, given a hierarchical structure of a graph, one can figure out the complex graph system from specific representations of objects to a high-level abstraction in a bottom-up manner. It is worth noting that this intuition is consistent with the idea of encoding graphs in the structural information theory. Having solved the problem of quantifying structural information, Li and Pan [15] proposed encoding the graph to an HPT. To facilitate comprehension and further analysis, we recall the definition of HPT.

Definition 1. Hierarchical Partition Tree (HPT). Let $G = (V, E)$ be an undirected and unweighted graph. Each node $\alpha \in T$ in the hierarchical partition tree T of G is associated with a non-empty set T_α satisfying the following properties:

- (1) The root node λ is associated with the node set of the entire graph, i.e., $T_\lambda = V$. Then the codeword of the whole graph is $c(V) = \lambda$.
- (2) Each non-leaf node α is the codeword of node subset $T_\alpha \subseteq V$, i.e., $c(T_\alpha) = \alpha$. Suppose that its immediate successor nodes are $S(\alpha) = \{\beta_1, \beta_2, \dots, \beta_s\}$. Then, $\{T_{\beta_1}, T_{\beta_2}, \dots, T_{\beta_s}\}$ is a partition of T_α , i.e., $\bigcup_{i=1}^s T_{\beta_i} = T_\alpha$ and $T_{\beta_i} \cap T_{\beta_j} = \emptyset, \forall i \neq j$.
- (3) Each leaf node γ is the codeword of one single node $v \in V$, i.e., $c(\{v\}) = \gamma$ and $T_\gamma = \{v\}$.

Based on the definition of HPT, the overview of Hi-PART has been illustrated in Figure 1. Hi-PART is encapsulated in a two-step procedure containing: (1) the HPT construction; and (2) graph summarization. We first introduce the structure entropy, which measures the amount of graph information preserved by HPT. Then, based on the structure entropy, two operations on HPT and a greedy algorithm are provided for optimization. Finally, GNNs are employed to learn the graph representation via the optimized HPT in the graph summarization stage. We summarize the main notions in Table 1.

4.1 Information Quantification for HPT

Generally, given a graph, there are numerous corresponding HPTs. Therefore, it leaves open an important question: "what makes for a good HPT for graph representation learning?". Here, we consider this problem from the perspective of structural information theory. Since tree is a simpler data structure, it is expected that a certain amount of information is lost while encoding graphs to trees. Thus a high-quality HPT should retain a maximum possible structural information. In

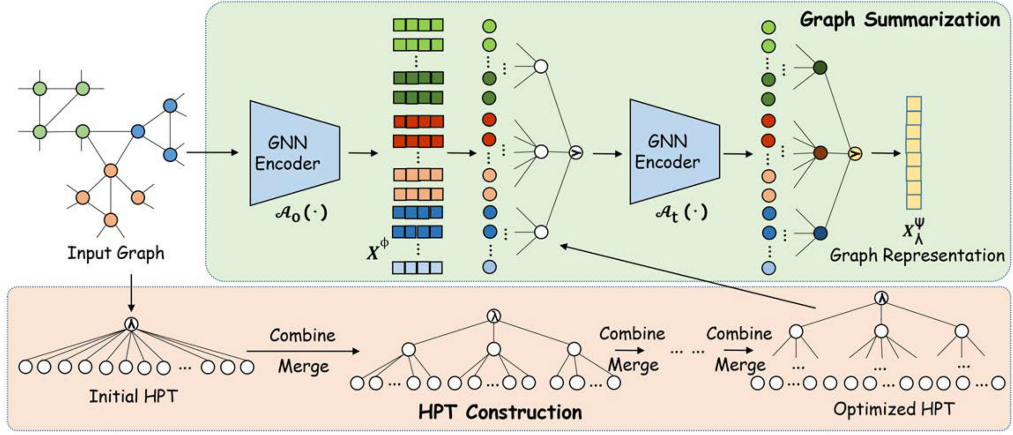


Fig. 1. The overview of Hi-PART. Given an input graph, we acquire its graph representation from the following two stages. In the HPT construction stage, the HPT is hierarchically constructed via the combining and merging operators; in the graph summarization stage, a GNN encoder $\mathcal{A}_0$ is used to obtain the deep node representations X^ϕ , which can also be regarded as the leaf node features of HPT. Finally, another GNN encoder $\mathcal{A}_t$ is applied to summarize leaf node representations to the root node representation X_λ^ψ , i.e., the graph representation.

Table 1. Summary of Main Notations

Notation	Meaning	Notation	Meaning
G	Input graph	X	initial node features
V	node set of graph	E	edge set of graph
T	hierarchical partition tree	X'	node features of HPT
V'	node set of HPT	E'	edge set of HPT
$\mathcal{H}(G)$	structural entropy of graph G	$\mathcal{H}_k(G)$	k -structural entropy of graph G
k	maximum height of HPT	$\mathcal{A}(\cdot)$	GNN model
X^ϕ	learned node embeddings of graph	X^ψ	learned node embeddings of HPT
X_λ^ψ	learned graph embedding	$\mathcal{N}(v)$	neighbourhood nodes of node v
$f(\cdot)$	aggregation function of GNN	$\phi(\cdot)$	updating function of GNN

essence, the HPT provides a hierarchical encoding of the graph by first dividing the graph into communities and then encoding them. These resultant communities are then divided into subcommunities, which keeps on going. In this level-by-level encoding procedure, the entropy of HPT $\mathcal{H}^T(G)$ is the entropy sum of all the non-root nodes, which can be formally defined as follows:

$$\mathcal{H}^T(G) = - \sum_{\alpha \in T, \alpha \neq \lambda} \frac{g_\alpha}{\text{vl}(G)} \cdot \log_2 \frac{\text{vl}(\alpha)}{\text{vl}(\alpha^-)} \quad (5)$$

Here, $\text{vl}(\alpha)$ denotes the volume of T_α which is the sum of node degree in T_α and α^- denotes the parent node of α . For each node α , the probability of random walk from $v \in T_{\alpha^-}$ to $v \in T_\alpha$ is $\text{vl}(\alpha)/\text{vl}(\alpha^-)$. Therefore, the entropy of determining T_α from T_{α^-} is $-\log_2 \frac{\text{vl}(\alpha)}{\text{vl}(\alpha^-)}$. g_α is the number of edges between T_α and $V - T_\alpha$. Hence the probability of random walk from the nodes in $V - T_\alpha$ to those in T_α is $\frac{g_\alpha}{\text{vl}(G)}$. As proved in [15], given any node pair (v_1, v_2) and the assumption that $c(v_1)$ is known, $\mathcal{H}^T(G)$ is the expected amount of information necessary to determine $c(v_2)$ from random

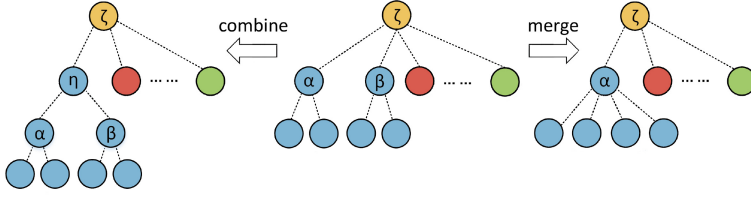


Fig. 2. An example of combining and merging operators.

walk. Furthermore, the structure entropy is the corresponding entropy of the HPT reserving the most information of the graph. Let $\mathcal{T}$ denote the set of all the HPTs. The structure entropy is defined as follows:

$$\mathcal{H}(G) = \min_{T \in \mathcal{T}} \{\mathcal{H}^T(G)\} \quad (6)$$

where T ranges over all the HPTs. If we limit the maximum height of HPT as k , the k -dimensional structure entropy is defined as follows:

$$\mathcal{H}_k(G) = \min_{T_k} \{\mathcal{H}^{T_k}(G)\} \quad (7)$$

where T_k ranges over all the HPTs with a maximum height of k . In this article, we choose to optimize the k -dimensional structure entropy. Since the distance from any leaf node to the root is less or equal to k , it is easier to determine the number of GNN layers for summarizing node features to the graph feature (see Section 4.3 for details).

4.2 HPT Construction

After defining the k -dimensional structure entropy, our objective is its optimization and obtaining the corresponding HPT. To the best of our knowledge, there exists no algorithm capable of achieving the optimal HPT in polynomial time. Therefore, we apply a greedy algorithm taking both efficiency and efficacy into account. We start with the most straightforward HPT, T , with all the nodes directly connected to the root node. We then provide two operators including combining and merging, leading to a new HPT T' with $\mathcal{H}^{T'}(G) < \mathcal{H}^T(G)$.

As shown on the left panel in Figure 2, the combining operator creates a new parent node, η , for two sister nodes, α and β , whose original parent node is ζ . After combining, the parent node of η is ζ and the associated node set of η is $T_\eta = T_\alpha \cup T_\beta$. Intuitively, the combining operator creates a new community including two nodes (sets) and thus introduces more structural information. Let T and T' denote the original and changed HPTs, respectively. The amount of reduced entropy is

$$\begin{aligned} \Delta_G^C(T; \alpha, \beta, \zeta) &= \mathcal{H}^T(G) - \mathcal{H}^{T'}(G) \\ &= \frac{C_{\alpha, \beta}}{\text{vl}(G)} \cdot \log_2 \frac{\text{vl}(\zeta)}{\text{vl}(\alpha) + \text{vl}(\beta)} \end{aligned} \quad (8)$$

where $C_{\alpha, \beta} = (g_\alpha + g_\beta - g_\eta)$ denotes the number of edges linked between node sets T_α and T_β . The combination of two node sets T_α and T_β can reduce more entropy if they have more edges linked to each other and smaller volume, which is in line with the property of community.

Unlike the combining operator, the merging operator merges two sister nodes α and β to α without creating new nodes. As shown on the right panel of Figure 2, after merged, β is deleted and the parent of all the immediate successors of β is changed to α . If β is a leaf node, it becomes its only successor without changing the entropy of the tree. Intuitively, the merging operator joins a node (set) to an existing community. The lowered entropy brought by the merging operation is calculated as follows:

$$\begin{aligned}
\Delta_G^M(T; \alpha, \beta, \zeta) &= \mathcal{H}^T(G) - \mathcal{H}^{T'}(G) \\
&= \frac{C_{\alpha, \beta}}{\text{vl}(G)} \log_2 \frac{\text{vl}(\zeta)}{\text{vl}(\alpha) + \text{vl}(\beta)} - \frac{I_\alpha}{\text{vl}(G)} \log_2 \frac{\text{vl}(\alpha) + \text{vl}(\beta)}{\text{vl}(\alpha)} \\
&\quad - \frac{I_\beta}{\text{vl}(G)} \log_2 \frac{\text{vl}(\alpha) + \text{vl}(\beta)}{\text{vl}(\beta)}
\end{aligned}$$

where $I_\alpha = (\sum_{a \in S(\alpha)} g_a) - g_\alpha$ denotes the number of edges linked among the immediate successor nodes of α and similar for I_β . If the node set α is fixed, the merging operator preferentially merges the node (set) that is most connected to α . Particularly, if α and β are both leaf nodes, the merging operator is the same as the combining operator.

ALGORITHM 1: k-dimensional structure entropy optimization

Input: Graph $G = (V, E)$ and k

Output: Hierarchical partition tree T and $\mathcal{H}^T(G)$

```

1 Construct the initial partition tree  $T$  with depth  $d = 2$ ;
2  $H_1 \leftarrow \mathcal{H}^T(G)$ ;
3  $\Delta_H \leftarrow 0$ ;
4 for  $d = 2 \rightarrow k - 1$  do
5   while there exist sister leaf nodes  $\alpha, \beta$  with  $\Delta_G^C(T; \alpha, \beta, \zeta) > 0$  and  $\text{depth}(\alpha) < d + 1$  do
6     Combine  $\alpha$  and  $\beta$  with the maximum  $\Delta_G^C(T; \alpha, \beta, \zeta)$  to  $\eta$ ;
7      $\Delta_H \leftarrow \Delta_H + \Delta_G^C(T; \alpha, \beta, \zeta)$ ;
8     while there exists leaf node  $\gamma$  with  $\Delta_G^M(T; \eta, \gamma, \zeta) > 0$  do
9       Merge  $\eta$  and  $\gamma$  with the maximum  $\Delta_G^M(T; \eta, \gamma, \zeta)$  to  $\eta$ ;
10       $\Delta_H \leftarrow \Delta_H + \Delta_G^M(T; \eta, \gamma, \zeta)$ ;
11    end
12  end
13 end
14  $H^T \leftarrow H_1 - \Delta_H$ ;
15 return  $T, H^T$ 

```

Given these two operators, we then present an algorithm for optimizing $\mathcal{H}_k(G)$ and obtaining the corresponding HPT. As presented in Algorithm 1, given the input graph G and the maximum height k of the HPT, we first construct the initial tree with all the leaf nodes directly connected to the root node and calculate its entropy (lines 1–3). Then, we deepen the depth of T layer by layer (line 4). To this end, the structure of T is iteratively updated in a greedy manner. In each iteration, the two most connected sister leaf nodes are selected to create a new community (lines 5–7) while not exceeding the current height limit $d + 1$. Then, we merge all the other leaf nodes strongly associated with this new community (lines 8–11). By repeating this process, the algorithm finally outputs the HPT T and its corresponding entropy $\mathcal{H}^T(G)$.

Time complexity. Since k is a constant number, we only need to consider the complexity of combining and merging operations in deriving T_{l+1} from T_l . The complexity of combining operations is $O(|E| \log |E|)$ since the selection of α and β can be implemented by a priority queue. Due to the traversal of neighboring nodes of nodes in η , the worst complexity of one merging operation is $O(|V|)$. And we need at most $|V|$ times of merging operations since each time one leaf node is transferred to the deeper layer. Thus the worst complexity of merging operations is $O(|V|^2)$ and the overall complexity of Algorithm 1 is $O(|E| \log |E| + |V|^2)$.

However, large graphs in the real world tend to be sparse. For a tightly connected community η , the number of edges between T_η and $V - T_\eta$ is much smaller than $|V|$, i.e., $g_\eta \ll |V|$. In this case, the complexity of Algorithm 1 is nearly $O(|E| \log |E|)$. Thus Algorithm 1 can be also well scaled to large sparse graphs.

4.3 Graph Summarization

The key object of graph summarization is to learn the graph representation based on the graph and its corresponding HPT. In order to fully utilize the graph structure, GNN is applied firstly on original graphs to learn the node representations aggregating the neighbourhood information. Formally, given an input graph $G = (V, E)$ with node feature matrix $X \in \mathcal{R}^{|V| \times d}$, a GNN encoder $\mathcal{A}_o(\cdot) : \mathcal{R}^{|V| \times d} \times \mathcal{R}^{|V| \times |V|} \rightarrow \mathcal{R}^{|V| \times d_1}$ is used to obtain the encoded node feature matrix $X^\phi \in \mathcal{R}^{|V| \times d_1}$.

Next, we aim to hierarchically summarize node representations to high-level graph representation in a bottom-up manner. To this end, we perform message passing on the optimized HPT. In particular, we regard the HPT $T = (V', E')$ as an undirected graph and use encoded node features X^ϕ as the raw features of leaf nodes in T . Then, the raw features X'_v for $v \in V'$ are as follows:

$$X'_v = \begin{cases} X^\phi_v, & v \in V \\ \mathbf{0}, & \text{otherwise} \end{cases} \quad (9)$$

Given an HPT $T = (V', E')$ with raw features X' , another GNN encoder $\mathcal{A}_t(\cdot) : \mathcal{R}^{|V'| \times d_1} \times \mathcal{R}^{|V'|^2} \rightarrow \mathcal{R}^{|V'| \times d_2}$ with at least k layers is used to acquire the tree node features $X^\psi \in \mathcal{R}^{|V'| \times d_2}$. Initially, only the leaf node features are non-zero. GNN aggregates them to update their parent node features, implying community representations. Through bottom-up learning, the root node feature X^ψ_λ , i.e., the graph representation, is finally derived from its child community representations.

After deriving the graph representation, we connect the GNN $\mathcal{A}_t$ with an MLP for different graph-level tasks. For example, the **negative log-likelihood (NLL)** loss function is used for graph classification, which is defined as follows:

$$NLL_{\text{loss}} = -\frac{1}{N} \sum_{j=1}^N \log(p_j(y_j)) \quad (10)$$

where $p_j(y_j)$ denotes the predicted probability of the ground-truth label y_j , and N is the batch size.

5 THEORETICAL ANALYSIS

Recently, Xu et al. [31] presented a theoretical framework for analyzing GNN's expressive power of capturing non-isomorphic graph structures. Here, we follow their approach to analyze Hi-PART's expressive power. We start by briefly reviewing the power of GNNs, highlighting the relationship between the WL test and GNNs.

THEOREM 1. *Given any two non-isomorphic graphs G_1 and G_2 , if a GNN $\mathcal{A} : \mathcal{G} \rightarrow \mathcal{R}^d$ maps G_1 and G_2 to different embeddings, the Weisfeiler–Lehman graph isomorphism test judges G_1 and G_2 as non-isomorphic.*

Theorem 1 indicates the upper limit of GNN's expressive power when capturing different graph structures. As shown in [31], there exist GNNs as powerful as the WL test, subject to the constraints in Theorem 2 being satisfied.

THEOREM 2. *Let $\mathcal{A} : \mathcal{G} \rightarrow \mathcal{R}^d$ be a GNN with a sufficient number of layers. $\mathcal{A}$ maps any graphs G_1 and G_2 judged as non-isomorphic by the Weisfeiler–Lehman test to different embeddings if the following conditions hold:*

- (a) $\mathcal{A}$'s aggregation function f and updating functions ϕ are injective.
- (b) $\mathcal{A}$'s graph-level readout function is injective.

The proofs of Theorems 1 and 2 are provided in [31]. Here, we define the class of GNNs satisfying the constraints in Theorem 2 as **Injective GNN (I-GNN)** and analyze Hi-PART's power using I-GNN as the backbone. It is first shown that Hi-PART can completely maintain I-GNN's power. Note that message passing on the original graphs keeps the power of I-GNN. Therefore it suffices to show that the message passing on the HPT does the same. Let an HPT $T = (V', E', X')$, where (V', E') denotes the structure of the tree and X' signifies the initial node embeddings. We illustrate that Hi-PART outputs the same embedding for two graphs if and only if the corresponding two HPTs are isomorphic.

THEOREM 3. *Assume the input feature space $\mathcal{X}$ is countable. Given two hierarchical partition trees $T_1 = (V'_1, E'_1, X'_1)$, $T_2 = (V'_2, E'_2, X'_2)$ with root nodes r_1, r_2 and an I-GNN $\mathcal{A}$ with a sufficient number of layers L , the root node representations output by $\mathcal{A}$ satisfy $h_{r_1}^{(L)} = h_{r_2}^{(L)}$ iff there exists a bijective function $\varphi : V'_1 \rightarrow V'_2$, s.t.,*

- (a) $\forall v \in V'_1, X'_{1,v} = X'_{2,\varphi(v)}$.
- (b) $\forall v, v' \in V'_1, (v, v') \in E'_1 \Rightarrow (\varphi(v), \varphi(v')) \in E'_2$.

PROOF. We prove Theorem 3 from the sufficiency and necessity, respectively. The sufficiency of Theorem 3 is obvious due to the isomorphism of T_1 and T_2 under this condition. Next, we prove the necessity. To begin with, we state the following lemma.

LEMMA 1. *Given two hierarchical partition trees T_1, T_2 and an I-GNN $\mathcal{A}$, suppose that the representation of node v in the l -th layer of $\mathcal{A}$ is $h_v^{(l)}$. Then for any nodes $v_1 \in V'_1$ and $v_2 \in V'_2$, $h_{v_1}^{(l)} = h_{v_2}^{(l)} \Rightarrow h_{v_1}^{(i)} = h_{v_2}^{(i)}, 0 \leq i \leq l$.*

PROOF. Recall that

$$h_v^{(i)} = \phi \left(h_v^{(i-1)}, f \left(\{ h_u^{(i-1)} : u \in \mathcal{N}(v) \} \right) \right), \quad 1 \leq i \leq l \quad (11)$$

According to the constraints in Theorem 2, ϕ is an injective function in I-GNN. Thus we have $h_{v_1}^{(i)} = h_{v_2}^{(i)} \Rightarrow h_{v_1}^{(i-1)} = h_{v_2}^{(i-1)}$. Then, Lemma 1 is proved by induction. $\square$

The key of proof for Theorem 3 is to construct the bijective function $\varphi : V'_1 \rightarrow V'_2$ satisfying the two conditions, initialized by $\varphi(r_1) = r_2$. To this end, we first construct the bijective mapping between the nodes in $\mathcal{N}(r_1)$ and $\mathcal{N}(r_2)$. Then, for each pair of nodes in the mapping, we construct the bijective mapping between their immediate successor nodes by induction.

(i) Recall that

$$\begin{aligned} h_{r_1}^{(L)} = h_{r_2}^{(L)} &\Rightarrow \phi \left(h_{r_1}^{(L-1)}, f \left(\{ h_u^{(L-1)} : u \in \mathcal{N}(r_1) \} \right) \right) \\ &= \phi \left(h_{r_2}^{(L-1)}, f \left(\{ h_u^{(L-1)} : u \in \mathcal{N}(r_2) \} \right) \right) \end{aligned}$$

Because f and ϕ are both injective, there exists a bijective function $\varphi_{r_1} : \mathcal{N}(r_1) \rightarrow \mathcal{N}(r_2)$, s.t., $\forall u \in \mathcal{N}(r_1)$,

- (a) $h_u^{(L-1)} = h_{\varphi_{r_1}(u)}^{(L-1)}$ and $h_u^{(0)} = h_{\varphi_{r_1}(u)}^{(0)}$.
- (b) $(u, r_1) \in E'_1 \Rightarrow (\varphi_{r_1}(u), \varphi(r_1)) \in E'_2$.

And according to Lemma 1, we have $h_{r_1}^{(L-1)} = h_{r_2}^{(L-1)}$ and $h_{r_1}^{(0)} = h_{r_2}^{(0)}$, i.e., $X'_{1,r_1} = X'_{2,r_2}$.

(ii) For any non-root and non-leaf nodes $v_1 \in V'_1$, $v_2 \in V'_2$ satisfying $h_{v_1}^{(l)} = h_{v_2}^{(l)}$, we have

$$\begin{aligned} & \phi \left(h_{v_1}^{(l-1)}, f \left(\left\{ h_u^{(l-1)} : u \in \mathcal{N}(v_1) \right\} \right) \right) \\ &= \phi \left(h_{v_2}^{(l-1)}, f \left(\left\{ h_u^{(l-1)} : u \in \mathcal{N}(v_2) \right\} \right) \right) \end{aligned} \quad (12)$$

Also due to the injective properties of f and ϕ , and $h_{v_1}^{(l)} = h_{v_2}^{(l)}$, there exists a bijective function $\varphi_{v_1} : \mathcal{N}(v_1) - \{v_1^-\} \rightarrow \mathcal{N}(v_2) - \{v_2^-\}$, s.t., $\forall u \in \mathcal{N}(v_1) - \{v_1^-\}$,

- (a) $h_u^{(l-1)} = h_{\varphi_{v_1}(u)}^{(l-1)}$ and $h_u^{(0)} = h_{\varphi_{v_1}(u)}^{(0)}$.
- (b) $(u, v_1) \in E'_1 \Rightarrow (\varphi_{v_1}(u), \varphi_{v_1}(v_1)) \in E'_2$.

And according to Lemma 1, we have $h_{v_1}^{(l-1)} = h_{v_2}^{(l-1)}$ and $h_{v_1}^{(0)} = h_{v_2}^{(0)}$, i.e., $X'_{1,v_1} = X'_{2,v_2}$.

According to (i) and (ii), for each non-root node $v \in V'_1 - \{r_1\}$, there exists a bijective function φ_{v^-} that maps v to $\varphi_{v^-}(v) \in V'_2 - \{r_2\}$ satisfying

- (a) $h_v^{(0)} = h_{\varphi_{v^-}(v)}^{(0)}$
- (b) $(v, v^-) \in E'_1 \Rightarrow (\varphi_{v^-}(v), \mathbb{1}_{\{v^-=r_1\}}\varphi(v^-) \cup \mathbb{1}_{\{v^-\neq r_1\}}\varphi(v^-)(v^-)) \in E'_2$.

Thus we can construct the bijective function $\varphi : V'_1 \rightarrow V'_2$ by merging all the bijective functions φ_{v^-} , $v \in V'_1 - \{r_1\}$. $\square$

Following Theorem 3, we can draw the inference that Hi-PART can fully reserve the power of I-GNN.

COROLLARY 1. *Given any two non-isomorphic graphs G_1 and G_2 , if an I-GNN $\mathcal{A}$ maps G_1 and G_2 to different embeddings, Hi-PART with I-GNN also maps G_1 and G_2 to different embeddings.*

PROOF. For G_1 and G_2 , after L_o layers of message passing, consider the multisets $\{h_{v_1}^{(L_o)}\}, \{h_{v_2}^{(L_o)}\}$ denoting the leaf node embeddings in the two corresponding HPTs T_1, T_2 . Let r_1 and r_2 be the root nodes of T_1 and T_2 . There exist two conditions:

- (i) $\{h_{v_1}^{(L_o)}\} = \{h_{v_2}^{(L_o)}\}$, which means that G_1 and G_2 cannot be distinguished by an I-GNN.
- (ii) $\{h_{v_1}^{(L_o)}\} \neq \{h_{v_2}^{(L_o)}\}$. Then according to Theorem 3, after sufficient L_t layers of message passing on HPT, we have $h_{r_1}^{(L_t)} \neq h_{r_2}^{(L_t)}$.

According to (i) and (ii), Corollary 1 is proved. $\square$

Then, we reveal that Hi-PART with I-GNN can also recognize certain non-isomorphic graphs beyond the capabilities of the WL test and I-GNN.

THEOREM 4. *There exist certain non-isomorphic graphs G_1 and G_2 , s.t., the WL test judges G_1 and G_2 as isomorphic while Hi-PART with I-GNN maps them to different embeddings.*

PROOF. Figure 3 illustrates a typical example of two non-isomorphic graphs decided as isomorphic by the WL test but mapped to different embeddings by Hi-PART. Although the leaf node feature distributions in the two HPTs are the same, the HPT optimization algorithm maps the two graphs to non-isomorphic HPTs, violating the constraints in Theorem 3. Thus the root node representations, i.e., graph representations learned by Hi-PART are different. $\square$

Hi-PART identifies all the non-isomorphic graphs within the capabilities of the WL test as well as certain ones for which the WL test fails. Therefore one can conclude that Hi-PART goes beyond the power of I-GNNs and the WL test.

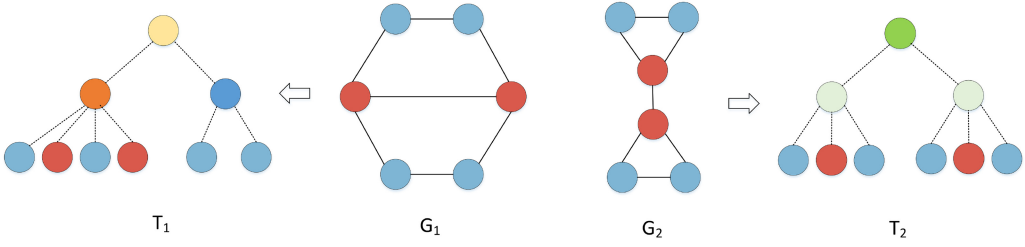


Fig. 3. An example of two non-isomorphic graphs decided as isomorphic by the WL test but discriminated by Hi-PART.

Table 2. Statistics and Properties of the Used Datasets

Dataset	#Nodes	#Edges	#Graphs	#Classes
COLLAB	74.5	2457.8	5000	3
IMDB-B	19.8	96.5	1000	2
IMDB-M	13.0	65.9	1500	3
REDDIT-B	429.6	497.8	2000	2
DD	284.3	1431.3	1178	2
PROTEINS	39.1	72.8	1113	2
MUTAG	17.9	19.8	188	2

6 EXPERIMENTAL STUDIES

In this section, we want to evaluate Hi-PART’s expressive power and generalization ability compared with graph pooling methods. More precisely, the experiments answer the following questions:

- **Q1.** How powerful is Hi-PART compared to GNN and graph pooling methods?
- **Q2.** Can Hi-PART depend on a single classification loss to ensure its training stability?
- **Q3.** How is the efficiency of Hi-PART?
- **Q4.** How does Hi-PART perform in graph classification?
- **Q5.** How do the hyper-parameters influence the performance of Hi-PART?
- **Q6.** How does the HPT structure influence the performance of Hi-PART?
- **Q7.** What is the visualization of the HPT structure?

6.1 Datasets and Experimental Settings

Datasets. We evaluate Hi-PART on seven graph classification benchmarks from the TU Dataset [18] including four social network datasets: COLLAB [33], IMDB-B, IMDB-M, REDDIT-B [32] and three biochemical network datasets: DD, PROTEINS [9], MUTAG [27]. The detailed statistics and properties are displayed in Table 2.

Baselines. We compare Hi-PART with five GNNs and several state-of-the-art graph pooling methods: (1) **GCN**. (2) **GAT**. (3) **GraphSAGE**. (4) **ID-GNN**. (5) **GIN**. GNNs [12, 13, 26, 31, 35] with mean or sum pooling providing an anchor of the performance. (6) **SortPool**. (7) **SAGPool**. (8) **TopKPool**. (9) **CGIPool**. The graph pooling methods [11, 14, 21, 38] based on node sampling that select the survived nodes using the importance scores. (10) **DiffPool**. (11) **ASAP**. (12) **MinCut-Pool**. (13) **HaarPool**. (14) **StructPool**. (15) **LCPN**. The graph pooling methods [4, 22, 25, 28, 34, 36] based on node clustering that groups nodes into clusters using the learnable cluster assignment matrix. (16) **EdgePool**. The method [8] that gradually merges two adjacent nodes with a high score

edge. (17) **SOPool_{attn}**. (18) **SOPool_{bimap}**. (19) **GMT**. The global pooling methods [3, 29] based on second-order pooling and multi-head attention, respectively. In order to make fair comparisons with these baseline methods, we directly quote the results from the previous literature or their original papers.

Models and configurations. Unless special instruction, we use GCN as the encoder for original graphs and HPTs to ensure a fair comparison. We use one-hot embeddings of node labels as raw node features for biochemical networks. Following [11], we use one-hot embeddings of node degree as raw features. A three-layer MLP is used as the classifier. Dropout with a keep rate of 0.7 is applied to both GNN layers and MLP. The hyper-parameters we tune for each dataset are: (1) the batch size $\in \{16, 32\}$, (2) the number of hidden units $\in \{16, 32, 128\}$ (3) the maximum height of HPT $k \in \{2, 3\}$, (4) the number of GNN $\mathcal{A}_o$ layers $\in \{1, 2, 3\}$, (5) the number of GNN $\mathcal{A}_t$ layers $\in \{2, 3, 4\}$.

Following the standard settings [18] of the graph classification, we perform 10-fold cross validation and report the average of validation accuracies across the 10 folds within the cross-validation. Following GIN [31], a single epoch with the best cross-validation accuracy averaged over the 10 folds was selected. The above evaluation is repeated 10 times and standard deviations are reported over all 10 repetitions, and additionally across all one hundred runs. Our 10-fold splittings are exactly the same with TopKPool [11]. In order to make fair comparisons with these baseline methods, we directly quote the results from the previous literature or their original papers if available. If results are not previously reported, we implement them based on the public source codes and conduct a hyperparameter search according to the original paper.

6.2 Training Performance

Training set accuracy (Q1). To validate our theoretical analysis of Hi-PART's expressive power, we compare the training accuracy of Hi-PART-GIN with that of GIN and GMT. GIN and GMT are GNN and graph pooling methods provably as powerful as the WL test. As shown in [31], a powerful model should achieve high training accuracy. Figure 4 shows the training accuracy curves of each method in DD, IMDB-BINARY, and COLLAB datasets. It is seen that GIN and GMT achieve similar training accuracies since they are both as powerful as the WL test. Furthermore, Hi-PART consistently outperforms GIN and GMT on the three datasets. This confirms our theoretical analysis that Hi-PART can go beyond the power of the WL test.

Training stability (Q2). To validate the training stability, we draw the training loss curves of Hi-PART-GIN in Figure 5. It is seen that the training loss converges after 400 epochs for all the datasets except for MUTAG. The relative training instability for MUTAG might be attributed to the lack of training data (<200). In general, the results indicate the sufficiency of Hi-PART depending on a single classification loss for training stability.

Time efficiency (Q3). To validate the efficiency of Hi-PART, we compare the preprocessing time with training time on all datasets, which make up the total time consumption of Hi-PART. The results are shown in Figure 6. It is observed that Hi-PART only spends less than 5% additional training time on preprocessing in all datasets. This small increase does not put a great burden on Hi-PART's efficiency especially as it comes with a significant performance improvement compared to GNNs. Note that our preprocessing for each graph is independent. Such a property enables Hi-PART to employ distributed computing when processing a large number of graphs.

6.3 Test Performance (Q4)

6.3.1 Performance Comparison. Next, we compare the accuracies on the test set with all the baseline methods in Table 3. The results indicate that the performance of Hi-PART is superior to the SOTA on 6 out of 7 benchmarks. In particular, Hi-PART improves the performance by 3.8%

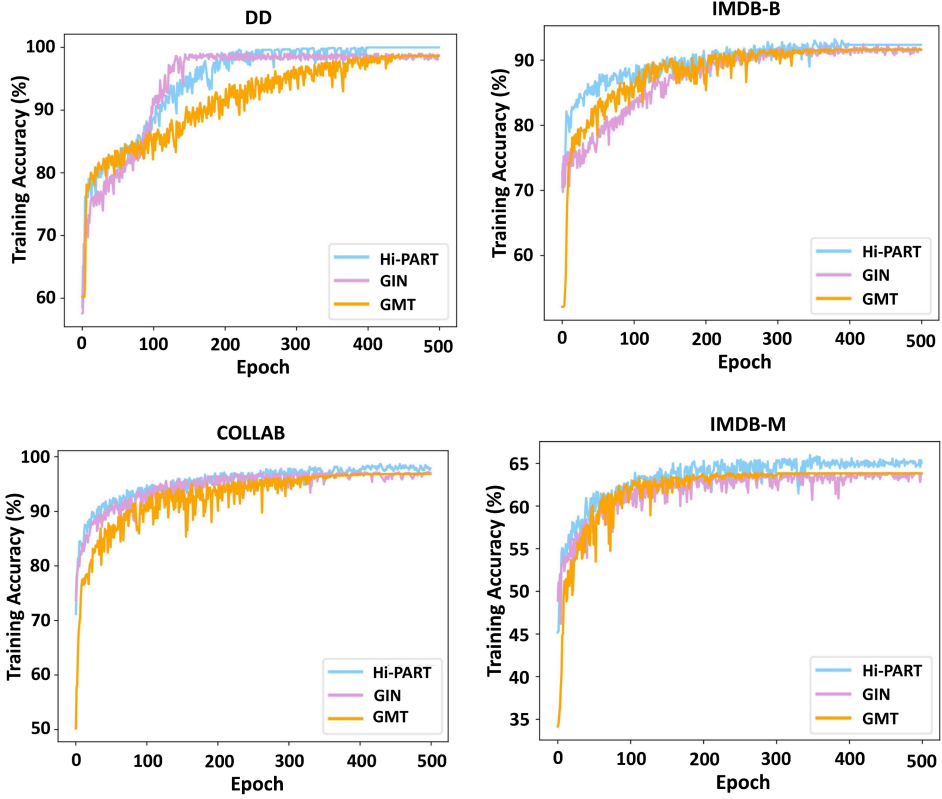


Fig. 4. Comparisons of Training accuracy.

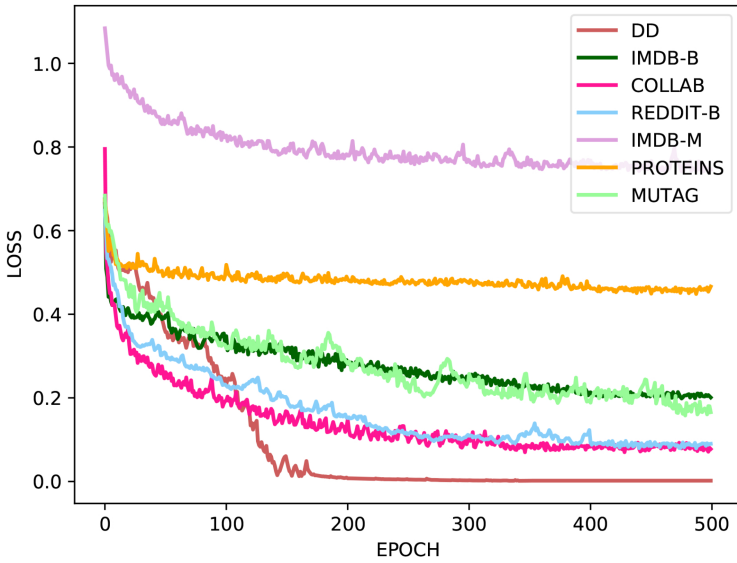


Fig. 5. Training loss of Hi-PART.

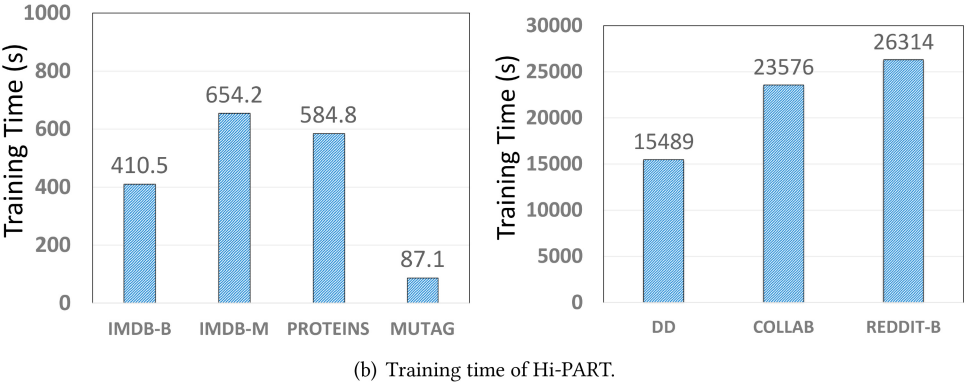
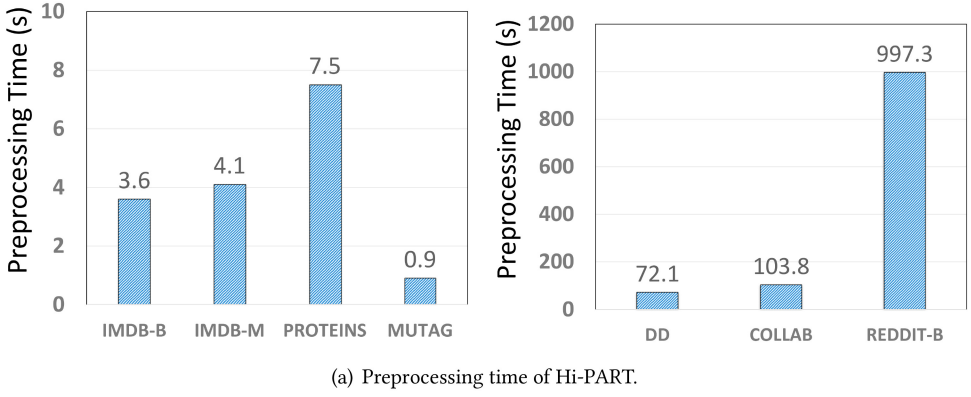


Fig. 6. Running time of Hi-PART.

and 1.9% over the best graph pooling baselines ASAP and MinCutPool on MUTAG and COLLAB datasets, which confirms the effectiveness of Hi-PART. Compared with all the baselines, Hi-PART is able to adequately leverage the graph structure information by converting it into a more effective HPT, thus achieving the best performance.

In addition to the widely applied accuracy metric, we also use macro-F1 score to test the performance of Hi-PART for more convincing comparisons. Specifically, we select the strong baselines from Table 3 and reproduce their codes with macro-F1 score. The results are shown in Table 4. From the results, we observe that Hi-PART outperforms the strong baselines on 6 out of 7 datasets, which is consistent with the results of accuracy and again validate the superior performance of Hi-PART.

6.3.2 Hi-PART with Other GNNs. To validate the extensibility of proposed Hi-PART framework, we also test Hi-PART on four GNN backbones including GAT [26], GraphSAGE [12], ID-GNN [35], and GIN [31]. We present the results in Table 5. We observe that Hi-PART shows significant improvements for each GNN model on all the datasets. In particular, the consistent superior performances of Hi-PART-GIN over GIN demonstrate that Hi-PART can enhance both the expressive power and generalization ability of GIN.

6.4 Hyper-Parameter Studies (Q5)

In this section, we study the impact of hyper-parameters on Hi-PART. Since the height of HPT k is the only extra hyper-parameter involved in Hi-PART, we first compare the performance for

Table 3. Comparison between Hi-PART and Other Methods in Terms of Graph Classification Accuracy (%)

Dataset	Social Network				Biochemical Network		
	COLLAB	IMDB-B	IMDB-M	REDDIT-B	DD	PROTEINS	MUTAG
GCN	78.6 ± 0.3	72.7 ± 6.4	50.4 ± 0.1	82.7 ± 2.2	78.1 ± 4.3	74.8 ± 2.8	76.6 ± 1.8
GAT	79.2 ± 2.1	73.9 ± 4.8	50.2 ± 1.2	84.4 ± 2.8	79.2 ± 5.1	75.0 ± 2.3	80.7 ± 2.6
GraphSAGE	77.8 ± 1.6	72.4 ± 4.6	50.8 ± 2.5	83.3 ± 1.9	78.8 ± 4.0	74.6 ± 1.9	81.1 ± 1.6
ID-GNN	81.1 ± 2.6	74.2 ± 5.4	51.7 ± 3.3	87.6 ± 2.9	80.8 ± 3.2	75.5 ± 1.8	88.6 ± 1.9
GIN	80.2 ± 1.9	75.1 ± 5.1	52.3 ± 2.8	92.4 ± 2.5	82.0 ± 2.7	76.2 ± 2.8	89.4 ± 5.6
SortPool	73.8 ± 0.5	70.0 ± 0.9	47.8 ± 0.9	74.5 ± 2.1	79.4 ± 0.9	75.5 ± 0.9	85.8 ± 1.7
SAGPool	73.3 ± 1.6	73.4 ± 2.9	51.1 ± 3.0	73.9 ± 2.6	76.5 ± 2.4	71.9 ± 1.0	82.9 ± 5.6
TopKPool	77.6 ± 2.1	75.4 ± 3.0	51.8 ± 3.7	85.5 ± 1.3	82.4 ± 2.9	77.7 ± 4.5	87.2 ± 7.8
CGIPool	76.4 ± 1.4	72.4 ± 0.9	51.5 ± 0.7	84.9 ± 1.7	80.3 ± 0.7	74.1 ± 2.3	86.6 ± 4.8
DiffPool	75.5 ± 2.6	75.9 ± 1.9	45.2 ± 3.1	86.8 ± 2.8	80.6 ± 3.2	77.6 ± 5.0	80.8 ± 5.3
ASAP	82.1 ± 0.3	77.9 ± 0.5	55.2 ± 0.3	92.0 ± 1.1	81.6 ± 0.4	78.3 ± 4.0	<u>91.3 ± 0.7</u>
MinCutPool	<u>83.4 ± 1.7</u>	77.8 ± 0.4	54.9 ± 0.2	91.4 ± 1.5	80.8 ± 2.3	76.5 ± 2.6	87.2 ± 2.1
HaarPool	81.4 ± 2.2	76.7 ± 0.6	53.0 ± 0.1	90.3 ± 1.8	79.8 ± 2.3	80.4 ± 1.8	90.0 ± 3.6
StructPool	74.2 ± 2.9	74.7 ± 2.8	52.5 ± 3.6	89.9 ± 2.5	<u>84.2 ± 3.3</u>	80.4 ± 2.9	90.4 ± 6.2
LPCN	–	73.0 ± 5.8	51.5 ± 4.0	–	<u>78.4 ± 3.5</u>	75.8 ± 4.2	–
EdgePool	67.1 ± 2.7	77.5 ± 0.3	54.7 ± 0.4	–	80.3 ± 0.4	79.6 ± 0.3	87.3 ± 1.2
SOPool _{attr}	80.2 ± 1.6	<u>78.9 ± 4.2</u>	54.8 ± 3.1	91.7 ± 2.7	82.6 ± 2.5	78.5 ± 2.8	90.0 ± 5.1
SOPool _{bimap}	80.4 ± 2.1	77.7 ± 4.5	54.5 ± 4.0	89.6 ± 3.3	81.1 ± 3.1	78.7 ± 3.1	90.9 ± 5.7
GMT	80.7 ± 0.5	78.4 ± 0.2	<u>55.1 ± 0.3</u>	<u>92.8 ± 3.5</u>	82.2 ± 0.4	<u>80.0 ± 0.2</u>	91.0 ± 0.8
Hi-PART	85.3 ± 0.9	79.9 ± 2.5	55.5 ± 2.2	93.9 ± 2.4	84.5 ± 2.7	79.5 ± 3.2	95.1 ± 6.1

The best and second-best results are highlighted with boldface and underlined. - denotes that there are no public record and code for the method on the dataset.

Table 4. Comparison between Hi-PART and Other Methods in Terms of Graph Classification Macro-F1 Score

Dataset	Social Network				Biochemical Network		
	COLLAB	IMDB-B	IMDB-M	REDDIT-B	DD	PROTEINS	MUTAG
GIN	76.9 ± 2.2	74.9 ± 4.5	50.4 ± 2.5	92.3 ± 2.2	81.2 ± 2.3	75.3 ± 2.5	88.6 ± 5.1
ASAP	<u>80.9 ± 0.5</u>	77.8 ± 0.8	53.2 ± 0.6	92.0 ± 1.1	81.0 ± 0.8	77.5 ± 4.4	<u>90.3 ± 1.2</u>
MinCutPool	80.5 ± 2.2	77.6 ± 0.8	52.4 ± 0.6	91.2 ± 1.4	80.1 ± 2.1	75.7 ± 2.9	86.1 ± 2.5
StructPool	71.8 ± 2.7	74.6 ± 2.5	50.8 ± 3.1	89.8 ± 2.8	<u>83.3 ± 3.7</u>	79.6 ± 2.4	89.7 ± 5.6
SOPool _{attr}	78.0 ± 1.5	<u>78.8 ± 4.0</u>	52.5 ± 3.4	91.6 ± 2.4	81.7 ± 1.8	77.6 ± 2.5	89.1 ± 4.7
GMT	78.8 ± 0.9	78.2 ± 0.4	<u>53.4 ± 0.5</u>	<u>92.7 ± 3.2</u>	81.4 ± 0.6	<u>79.1 ± 0.5</u>	90.4 ± 1.1
Hi-PART	83.1 ± 1.2	79.8 ± 1.9	54.0 ± 2.6	93.8 ± 2.5	83.8 ± 2.4	78.7 ± 2.7	94.5 ± 5.3

Table 5. Hi-PART with Different GNN Backbones

Dataset	Social Network				Biochemical Network		
	COLLAB	IMDB-B	IMDB-M	REDDIT-B	DD	PROTEINS	MUTAG
Hi-PART-GAT	85.5 ± 1.3	80.2 ± 2.1	55.7 ± 2.8	94.1 ± 2.3	84.9 ± 3.1	79.7 ± 2.4	95.3 ± 5.2
Hi-PART-SAGE	84.6 ± 1.3	79.7 ± 2.7	55.0 ± 2.7	94.1 ± 2.9	84.6 ± 3.1	79.2 ± 3.8	95.1 ± 5.4
Hi-PART-IDGNN	85.7 ± 1.9	80.2 ± 2.8	56.3 ± 3.1	94.6 ± 2.3	85.3 ± 2.4	80.5 ± 3.4	95.5 ± 5.8
Hi-PART-GIN	86.1 ± 2.1	80.5 ± 3.2	56.2 ± 2.9	94.5 ± 1.9	85.5 ± 3.1	80.2 ± 2.5	95.6 ± 4.8

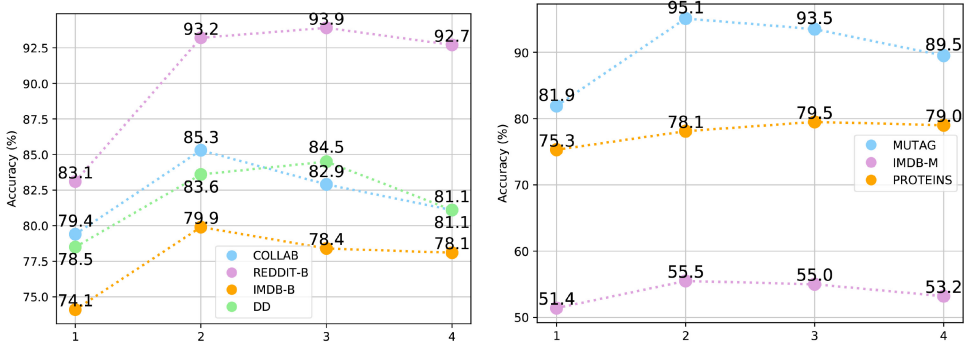
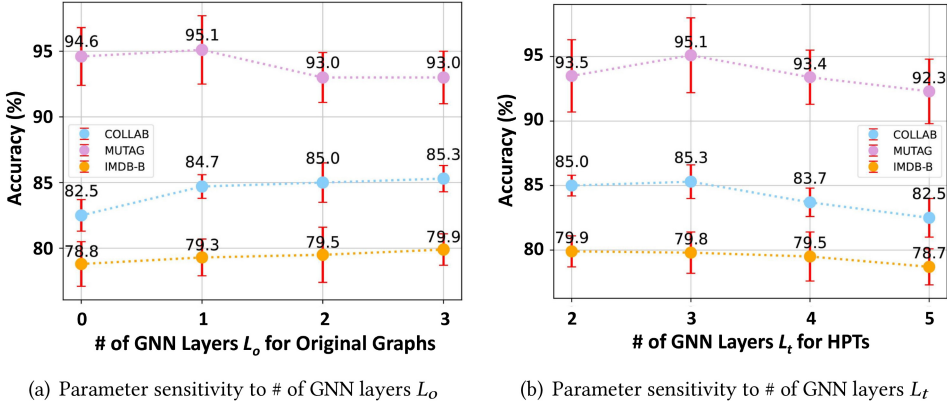
Fig. 7. Hi-PART with different HPT heights k .

Fig. 8. Hyper-parameter studies on Hi-PART.

different values of k . We undertake experiments on all the seven datasets with $k = 1, 2, 3, 4$ and present the results in Figure 7. The observations are as follows.

(1) For $k = 1$, HPT is constructed with all leaf nodes directly connected to the root node and fails to provide any structural information, leading to poor performance on all the datasets. (2) In small social network datasets with an average graph size of less than 100, such as IMDB-B, IMDB-M, and COLLAB, we find that HPT with a height of 2 effectively captures the graph structure; In large social network dataset REDDIT-B, Hi-PART benefits from finer-grained hierarchical partition of communities. However, the performances have a small degradation when the partition is too fragmented because some closely related people might be divided into different groups. (3) The performances on biochemical networks have a similar trend to the social networks according to the average graph size in the datasets. The main difference comes from the fact that the performances have a more obvious decrease when k is too large. The reason might be that too fine-grained partition might break up the groups of the molecule and thus destroy its properties. Overall the HPT height is the key hyperparameter in Hi-PART and should be tuned carefully according to the average graph size of the dataset.

In addition to k , we explore the influence of hyper-parameters in GNNs used for message passing on original graphs and HPTs. As analyzed in [16], a varied number of layers might significantly affect GNN's performance. Thus we conduct experiments to test Hi-PART's sensitivity to different numbers of GNN layers L_o and L_t for the original graphs and HPTs, respectively. Figures 8(a) and 8(b) show that Hi-PART's performances under different layer number settings are competitive

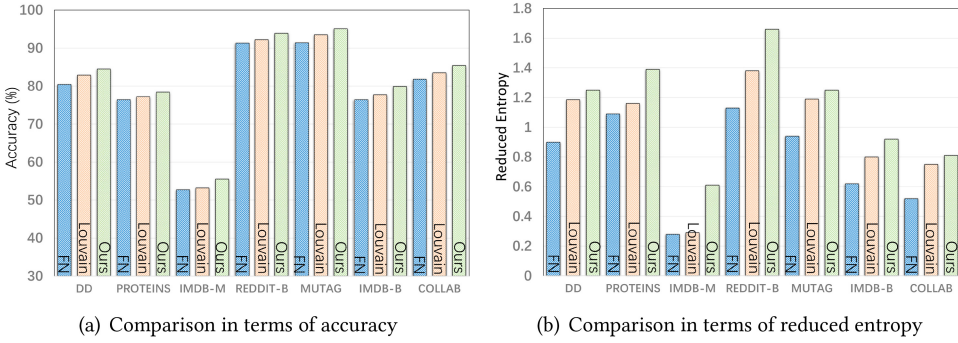


Fig. 9. Performance of Hi-PART with different HPT structures.

compared to the best results of baseline methods. It is also noteworthy that for $L_o = 0$, the original graphs are unseen by Hi-PART. In other words, Hi-PART only utilizes the HPTs and raw node features for graph representation learning. Although the classification accuracy is slightly lower, it is still competitive with the SOTA methods. This result again confirms that Hi-PART can effectively capture the structure information of graphs.

6.5 HPT Structure Analysis (Q6)

In this section, our objective is to explore the impact of different HPT structures, i.e., the amount of graph information retained by HPT, on the performance of Hi-PART. We compare the HPT optimization algorithm with two community detection algorithms, including FN [19] and Louvain [5]. These algorithms are widely regarded as the most effective algorithms in the field of community detection. To ensure fairness, we construct the HPTs with the same height as ours based on the community detection results by the baseline methods. For HPTs with heights larger than 2, we iteratively apply the two algorithms to divide large communities into more fine-grained ones. All the HPTs are embedded in our framework with the same hyper-parameter settings. We compare the three algorithms based on two criteria, including the average amount of reduced entropy and the graph classification accuracy.

The results are shown in Figure 9. It is observed that the proposed algorithm constructs the HPT with a consistent lower entropy than the others and achieves the highest classification accuracy on all the datasets. Since both FN and Louvain aim to optimize modularity [20], this result further justifies our choice of optimizing the structure entropy. The modularity is defined based on communities, whereas the structure entropy considers the graph's high-dimensional structural information from a hierarchical perspective. Therefore, the latter better fits with the task of graph representation learning. In general, for the same HPT depth, we find that the smaller entropy tends to better performances. And we leave the in-depth analyses of the relationship between HPT depth and GNN performances as a future work.

6.6 Visualization (Q7)

In order to facilitate a more intuitive and deep understanding of the optimized HPT structure, we randomly select one social network from the IMDB-M dataset and present the visualization of optimized HPTs of heights 2 and 3. The results are shown in Figure 10. We observe that with the entropy reduction, the hierarchical community structure of the graph is becoming increasingly clearer, which provides the GNN with key information from the tree structure. In addition, we find that the entropy reduction from the graph to 2-HPT is far more pronounced than that from 2-HPT to 3-HPT, which indicates that 2-HPT might be sufficient to provide community information.

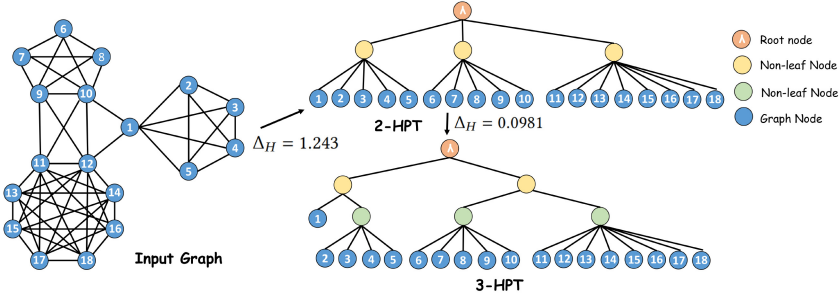


Fig. 10. Visualization of HPT.

This might also be a possible explanation for the slight performance degradation of 3-HPT on the IMDB-M dataset, which we will explore more deeply in our future work.

7 CONCLUSION

In this work, we propose a novel framework, Hi-PART, for graph-level representation learning. We first build the HPT that reserves the most information of the graph by optimizing the structure entropy. Then based on HPT's hierarchical structure, we apply GNNs to summarize node features to the graph feature in a bottom-up way. Furthermore, we theoretically demonstrate that Hi-PART goes beyond the power of the WL test in discriminating different graphs, thus being more powerful than existing graph pooling methods. Experimental results not only verify Hi-PART's pronouncing improvements compared to the SOTA graph pooling methods but also validate its expressive power and effectiveness in capturing graph structure information.

REFERENCES

- [1] Kartik Anand and Ginestra Bianconi. 2009. Entropy measures for networks: Toward an information theory of complex topologies. *Physical Review E* 80, 4 (2009), 045102.
- [2] László Babai. 2016. Graph isomorphism in quasipolynomial time. In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing*. 684–697.
- [3] J. Baek, M. Kang, and S. J. Hwang. 2021. Accurate learning of graph representations with graph multiset pooling. *The International Conference on Learning Representations (ICLR'21)*.
- [4] Filippo Maria Bianchi, Daniele Grattarola, and Cesare Alippi. 2020. Spectral clustering with graph neural networks for graph pooling. In *Proceedings of the International Conference on Machine Learning*. PMLR, 874–883.
- [5] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008.
- [6] Samuel L. Braunstein, Sibasis Ghosh, and Simone Severini. 2006. The Laplacian of a graph as a density matrix: A basic combinatorial approach to separability of mixed states. *Annals of Combinatorics* 10, 3 (2006), 291–317.
- [7] Jin-Yi Cai, Martin Fürer, and Neil Immerman. 1992. An optimal lower bound on the number of variables for graph identification. *Combinatorica* 12, 4 (1992), 389–410.
- [8] F. Diehl. 2019. Edge contraction pooling for graph neural networks. arXiv preprint arXiv:1905.10990.
- [9] Paul D. Dobson and Andrew J. Doig. 2003. Distinguishing enzyme structures from non-enzymes without alignments. *Journal of Molecular Biology* 330, 4 (2003), 771–783.
- [10] Francois Fouss, Alain Pirotte, Jean-Michel Renders, and Marco Saerens. 2007. Random-walk computation of similarities between nodes of a graph with application to collaborative recommendation. *IEEE Transactions on Knowledge and Data Engineering* 19, 3 (2007), 355–369.
- [11] Hongyang Gao and Shuiwang Ji. 2019. Graph U-Nets. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2083–2092.
- [12] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 1025–1035.
- [13] T. N. Kipf and M. Welling. 2016. Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.

- [14] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. 2019. Self-attention graph pooling. In *Proceedings of the International Conference on Machine Learning*. PMLR, 3734–3743.
- [15] Angsheng Li and Yicheng Pan. 2016. Structural information and dynamical complexity of networks. *IEEE Transactions on Information Theory* 62, 6 (2016), 3290–3339.
- [16] Meng Liu, Hongyang Gao, and Shuiwang Ji. 2020. Towards deeper graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 338–348.
- [17] Yao Ma, Suhang Wang, Charu C. Aggarwal, and Jiliang Tang. 2019. Graph convolutional networks with eigenpooling. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 723–731.
- [18] C. Morris, M. N. Kriege, F. Bause, et al. 2020. Tudataset: A collection of benchmark datasets for learning with graphs. arXiv preprint arXiv:2007.08663.
- [19] Mark E. J. Newman. 2004. Fast algorithm for detecting community structure in networks. *Physical Review E* 69, 6 (2004), 066133.
- [20] Mark E. J. Newman and Michelle Girvan. 2004. Finding and evaluating community structure in networks. *Physical Review E* 69, 2 (2004), 026113.
- [21] Yunsheng Pang, Yunxiang Zhao, and Dongsheng Li. 2021. Graph pooling via coarsened graph infomax. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2177–2181.
- [22] Ekagra Ranjan, Soumya Sanyal, and Partha Talukdar. 2020. ASAP: Adaptive structure aware pooling for learning hierarchical graph representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34, 5470–5477.
- [23] Nicolas Rashevsky. 1955. Life, information theory, and topology. *The Bulletin of Mathematical Biophysics* 17, 3 (1955), 229–235.
- [24] Claude Elwood Shannon. 1948. A mathematical theory of communication. *The Bell System Technical Journal* 27, 3 (1948), 379–423.
- [25] Zidong Su, Zehui Hu, and Yangding Li. 2021. Hierarchical graph representation learning with local capsule pooling. In *ACM Multimedia Asia*. 1–7.
- [26] P. Veličković, G. Cucurull, A. Casanova, et al. 2017. Graph attention networks. arXiv preprint arXiv:1710.10903.
- [27] Nikil Wale, Ian A. Watson, and George Karypis. 2008. Comparison of descriptor spaces for chemical compound retrieval and classification. *Knowledge and Information Systems* 14, 3 (2008), 347–375.
- [28] Yu Guang Wang, Ming Li, Zheng Ma, Guido Montufar, Xiaosheng Zhuang, and Yanan Fan. 2020. Haar graph pooling. In *Proceedings of the International Conference on Machine Learning*. PMLR, 9952–9962.
- [29] Z. Wang and S. Ji. 2020. Second-order pooling for graph neural networks. *IEEE Transactions on Pattern Analysis & Machine Intelligence* 1 (2020), 1–1.
- [30] Boris Weisfeiler and Andrei Leman. 1968. The reduction of a graph to canonical form and the algebra which appears therein. *NTI, Series 2*, 9 (1968), 12–16.
- [31] K. Xu, W. Hu, J. Leskovec, et al. 2018. How powerful are graph neural networks? arXiv preprint arXiv:1810.00826.
- [32] Pinar Yanardag and S. V. N. Vishwanathan. 2015. Deep graph kernels. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1365–1374.
- [33] Pinar Yanardag and S. V. N. Vishwanathan. 2015. A structural smoothing framework for robust graph comparison. In *Proceedings of the 28th International Conference on Neural Information Processing Systems*. 2134–2142.
- [34] Z. Ying, J. You, C. Morris, et al. 2018. Hierarchical graph representation learning with differentiable pooling. *Advances in Neural Information Processing Systems*. (2018), 31.
- [35] Jiaxuan You, Jonathan M. Gomes-Selman, Rex Ying, and Jure Leskovec. 2021. Identity-aware graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35, 10737–10745.
- [36] Hao Yuan and Shuiwang Ji. 2020. StructPool: Structured graph pooling via conditional random fields. In *Proceedings of the 8th International Conference on Learning Representations*.
- [37] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. In *Proceedings of the International Conference on Neural Information Processing Systems*. 5165–5175.
- [38] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. 2018. An end-to-end deep learning architecture for graph classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32.
- [39] J. Zhou, G. Cui, S. Hu, et al. 2020. Graph neural networks: A review of methods and applications. *AI Open*, 1 (2020), 57–81.

Received 1 July 2022; revised 22 May 2023; accepted 29 November 2023